

CS 104B – Advanced .NET Programming

Fall 2006

Bank Account Manager

Due date: 11/28/2006 - in class demo

(You may work in groups of two or individually for this programming assignment.)

The Problem

In this programming assignment, you are to implement a Centralized Multi-User Concurrent Bank Account Manager. The system has two important components:

1. User Application (Windows or Web Application)

User applications are either Windows-based or Web-based and are used to maintain and update bank accounts (checking, savings, brokerage accounts, CDs, IRAs, 401 (k) s, and so on) and bank and personal contact information. The operations that can be performed on an account are withdrawing funds, depositing funds, and viewing bank information. Additionally, the banks and personal contact information are integrated with the bank accounts. Additionally, interest accounts (savings, etc.) can accrue interest at a fixed rate.

2. Database for user application (Microsoft Access or SQL Server)

Tables:

- Bank Account
- Type of Banks
- Banks' Contact
- Personal Contact

3. Multi-tier Application

The user application is a multi-tier application—Graphical User Interface, Data Tier, and Database.

4. What you will submit

When you have finished implementing the complete assignment as described above, put all the code in a separate directory in the directory of your portable drive (/project). You are required to submit your solution and in-class demo of your project.

5. Grading policy for all programming assignments

Grading:

- Works correctly ----- 50%
- In-line documentation ----- 5%
- Design Document
 - Quality of design ----- 15%

- Thoroughness evaluation ----- 10%
- In class demo ----- 20%

6. UI Design Guidelines

Visual Studio 2005 includes user interface (UI) guidelines--consistent, predictable UI. Here are some general principles for user interface design and it's the nature of rules of thumb.

▪ **Visibility of system status**

The system should always keep users informed about what is going on, through appropriate feedback within a reasonable time.

▪ **Match between system and the real world**

The system should speak the users' language, with words, phrases and concepts familiar to the user, rather than system-oriented terms. Follow real-world conventions, making information appear in a natural and logical order.

▪ **User control and freedom**

Users often choose system functions by mistake and will need a clearly marked "emergency exit" to leave the unwanted state without having to go through an extended dialogue. Support undo and redo.

▪ **Consistency and standards**

Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform conventions.

▪ **Error prevention**

Even better than good error messages is a careful design, which prevents a problem from occurring in the first place. Either eliminate error-prone conditions or check for them and present users with a confirmation option before they commit to the action.

▪ **Recognition rather than recall**

Minimize the user's memory load by making objects, actions, and options visible. The user should not have to remember information from one part of the dialogue to another. Instructions for use of the system should be visible or easily retrievable whenever appropriate.

▪ **Flexibility and efficiency of use**

Accelerators -- unseen by the novice user -- often speed up the interaction for the expert user, allowing the system to cater to both inexperienced and experienced users. Allow users to tailor frequent actions.

▪ **Aesthetic and minimalist design**

Dialogues should not contain information that is irrelevant or rarely needed. Every additional unit of information in a dialogue competes with the relevant units and diminishes their relative visibility.

- **Help users recognize, diagnose, and recover from errors**

Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.

- **Help and documentation**

Even though it is preferable for the system to be usable without documentation, it may be necessary to provide help and documentation. Any such information should be easy to search, focused on the user's task, list concrete steps to be carried out, and not be too large.